

REQUIREMENT ANALYSIS DOCUMENT

NAILVERSE

Not just NAILS.. IT's the WHOLE MOOD

Online Nail Art & Nail Beauty Online Platform

WEEK - 1

1. Requirement Analysis

Understanding the Nailverse Business

Nailverse is an online nail-care booking platform that connects customers with nail technicians for services such as manicures, pedicures, nail art, and nail extensions. Below is a structured breakdown of the business process, customer data requirements, and entity definitions for system design and database modeling.

The Process Flow:

- Browse / Search Services: The customer searches for the nail service they want (e.g. gel manicure, acrylic extension, nail art).
- Select Technician & Slot: After choosing the service, the customer picks a preferred technician and an available time slot.
- Appointment Booked: The customer confirms the salon location (or home-service address) and books the appointment.
- Checkout / Payment: After booking, the system moves the customer to the payment section, then to the checkout process.
- Service Delivery: On the scheduled date, the technician performs the nail service for the customer.
- Review & Support: After the service is completed, the customer may leave a review and rating. For issues such as damage, refunds, or rebooking, the customer can raise a support request.

Browse / Search Services → Select Technician & Slot → Appointment Booked → Checkout / Payment → Service Delivery → Review & Support

Entities:

Entity	Description	Key Attributes	Key Relationships
Customer	End-user who browses services and books nail appointments.	customer_id (PK), first_name, last_name, email, phone, created_at	One-to-Many with Appointment, Review

Technician	Nail artist/technician who performs the services.	technician_id (PK), first_name, last_name, email, phone, specialty, rating	One-to-Many with Appointment
Service	A nail service offered by the salon (e.g. manicure, pedicure, nail art).	service_id (PK), name, description, price, duration_minutes, category	Many-to-One with Technician (default), Many-to-Many with Appointment (via AppointmentService)
Appointment	Booking instance representing a customer's scheduled visit.	appointment_id (PK), customer_id (FK), technician_id (FK), appointment_date, status (booked, completed, cancelled), total_amount	Many-to-One with Customer, Many-to-One with Technician, One-to-One with Payment, Many-to-Many with Service (via AppointmentService)
Payment	Monetary transaction tied to an appointment.	payment_id (PK), appointment_id (FK), payment_method (cash, card, UPI), status (success, failed, refunded), transaction_id, amount, payment_date	One-to-One with Appointment
Review	Feedback and rating submitted by a customer for a service/technician.	review_id (PK), customer_id (FK), technician_id (FK), rating (1-5), comment, created_at	Many-to-One with Customer, Many-to-One with Technician

Customer Table Creation:

```
SQL> create table customer(customer_id varchar2(20) Primary key, first_name varchar2(20),
2 last_name varchar2(20), email varchar2(30), phone number(10), created_at date);
```

Table created.

```
SQL> desc customer
```

Name	Null?	Type
CUSTOMER_ID	NOT NULL	VARCHAR2(20)
FIRST_NAME		VARCHAR2(20)
LAST_NAME		VARCHAR2(20)
EMAIL		VARCHAR2(30)
PHONE		NUMBER(10)
CREATED_AT		DATE

Insert Values:

```
SQL> INSERT INTO customer
2 VALUES ('CU-1001', 'Meera', 'Iyer', 'meera.iyer@nailverse.com', 9876543210,
3 TO_DATE('2023-02-14', 'YYYY-MM-DD'));
```

1 row created.

Using Substitution Variables:

```

SQL> INSERT INTO customer (customer_id, first_name, last_name, email, phone, created_at)
VALUES (
    '&cust_id',
    '&first_name',
    '&last_name',
    '&email',
    &phone_num,
    DATE '&created_date'
);
Enter value for cust_id: CU-1002
old 3: '&cust_id',
new 3: 'CU-1002',
Enter value for first_name: Kavya
old 4: '&first_name',
new 4: 'Kavya',
Enter value for last_name: Rao
old 5: '&last_name',
new 5: 'Rao',
Enter value for email: kavya.rao@nailverse.com
old 6: '&email',
new 6: 'kavya.rao@nailverse.com',
Enter value for phone_num: 8765432109
old 7: &phone_num,
new 7: 8765432109,
Enter value for created_date: 2024-09-23
old 8: DATE '&created_date'
new 8: DATE '2024-09-23'

1 row created.

```

Like this we can create as many rows we want.

Viewing the Data:

```
SQL> select * from customer;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE	CREATED_A
CU-1001	Meera	Iyer	meera.iyer@nailverse.com	9876543210	14-FEB-23
CU-1002	Kavya	Rao	kavya.rao@nailverse.com	8765432109	23-SEP-24
CU-1003	Ananya	Suresh	ananya.suresh@nailverse.com	9087654321	09-NOV-23
CU-1004	Priya	Nair	priya.nair@nailverse.com	8901234567	09-SEP-25

Update:

```
SQL> update customer set phone = 9870654312 where customer_id = 'CU-1004';
```

```
1 row updated.0
```

```
INSERT INTO customer
```

```
VALUES ('C-2002', 'MISHA', 'AGAN', 'mishaagan@gmail.com', 8794562130, TO_DATE('2025-11-24', 'YYYY-MM-DD'));
```